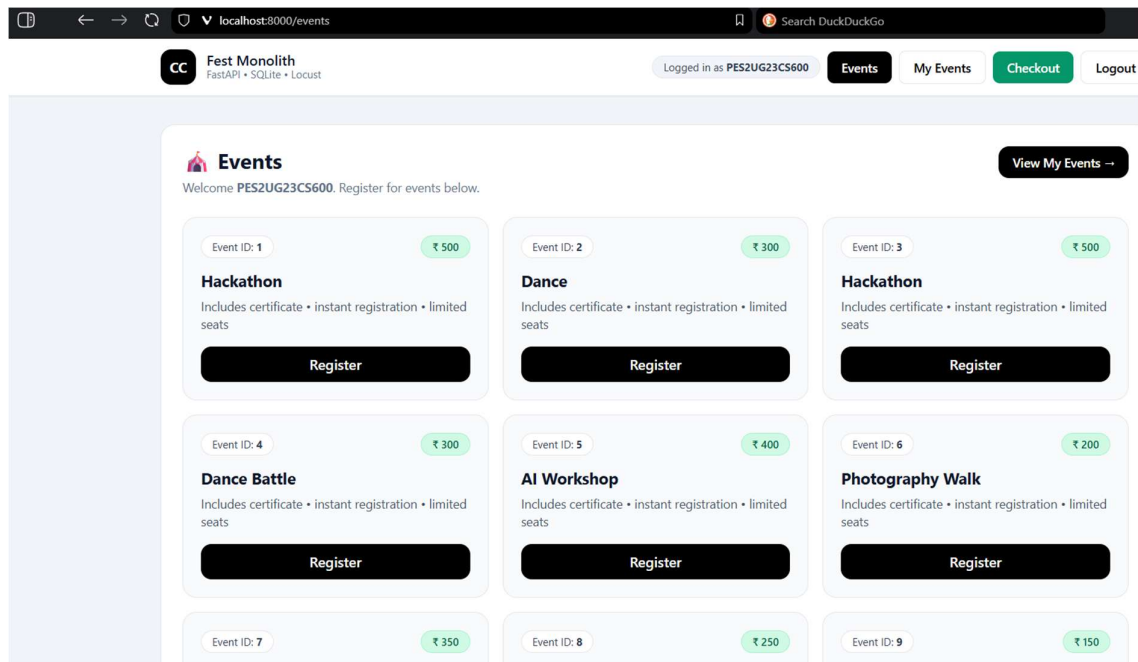


CC LAB2

SRIJAN DAS

PES2UG23CS600

SS1



SS2

CC

Fest Monolith
FastAPI • SQLite • Locust

Login

Create Account



Monolith Failure

HTTP 500

Error Message

division by zero

Why did this happen?

Because this is a **monolithic application**: all modules share the same runtime and deployment. When one feature crashes, it affects the whole system.

What should you do in the lab?

- Take a screenshot (crash demonstration)
- Fix the bug in the indicated module
- Restart the server and verify recovery

Back to Events

Login

CC Week X • Monolithic Applications Lab

SS3

CC

Fest Monolith

FastAPI • SQLite • Locust

Login

Create Account



Checkout

This route is used to demonstrate a monolith crash + optimization.

Total Payable

₹ 6600

☒ After fixing + optimizing checkout logic, re-run Locust and compare results.

What you should observe

- One buggy feature can crash the entire monolith.
- Inefficient loops cause high response times under load.
- Optimization improves performance but architecture still scales as one unit.

Next Lab: Split this monolith into Microservices (Events / Registration / Checkout).

CC Week X • Monolithic Applications Lab

SS4

The screenshot shows the Locust web interface with two main panels:

- Left Panel (Statistics):** Displays a table of performance metrics for GET requests to //checkout.

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)
GET	//checkout	2	0	4	2000	2000
	Aggregated	2	0	4	2000	2000

- Right Panel (Terminal):** Shows command-line execution details and server logs.

```

C:\Lab-2> .\env\Scripts> $ activate
1 # This file must be used with "source bin/activate" *from hash*

PROBLEMS OUTPUT TERMINAL ... powerShell - CC Lab-2 +v [ ] [x] [y]

(.venv) PS C:\Users\dshrij\cldg\sem6\cloudComputing\lab\lab2\PES2UG23CS600\Monolith_CC_Lab-2(CC Lab-2)> locust -f .\locust\checkout_locustfile.py

[2026-01-29T08:57:12Z] Srijan_Laptop/INFO/locust.main: Shutting Type Name
# reqs # fails | Avg Min Max Med | req/s

failures/s
-----|-----|-----|-----|-----|-----|-----|-----|
GET //checkout 2000 2000 2000 2000 2 2 0(0.00%) | 1017 3000 2000
2000 2000 2000 2000 2 -----|-----|-----|-----|-----|
000 2000 2000 2000 2000 2000 2
000 2000 2000 2000 2000 2000 2

-----|-----|-----|-----|-----|-----|-----|-----|
Aggregated 2000 2000 2000 2000 2000 2 2000 2000 2000 2000 2000
2000 2000 2000 2000 2000 2

(.venv) PS C:\Users\dshrij\cldg\sem6\cloudComputing\lab\lab2\PES2UG23CS600\Monolith_CC_Lab-2(CC Lab-2)> locust -f .\locust\checkout_locustfile.py
    
```

SS5

LOCUST

STATISTICSCHARTSFAILURES EXCEPTIONSCURRENT RATIO

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)
GET	//checkout	379	0	3	16	2000	56.64	2
Aggregated		379	0	3	16	2000	56.64	2

ABOUT

CC Lab-2 > checkout > _init_.py > ...

from database import get_db

PROBLEMS 3 OUTPUT TERMINAL

Type	Name	Avg	Min	Max	Med	req/s	failures/s	# reqs	# fails
GET	//checkout	56	1	2835	3	6.32	0.00	379	0
Aggregated		56	1	2835	3	6.32	0.00	379	0

Response time percentiles (approximated)

Type	Name	66%	75%	80%	90%	95%	98%	99%	99.9%	99.99%	100%	# reqs
GET	//checkout	3	3	3	4	16	2000	2000	2000	2000	2000	379
Aggregated		3	3	3	4	16	2000	2000	2000	2000	2000	379

SS6

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)
GET	//events?user=locust_user	196	0	280	2100	2400	377.38	106
Aggregated		196	0	280	2100	2400	377.38	106

ABOUT

Aggregated

4.92 196 0(0.00%) | 377 106 2412 280 | 0.00

Response time percentiles (approximated)

Type	Name	50%	66%	75%	80%	90%	95%	98%	99%	99.9%	99.99%	100%	# reqs
GET	//events?user=locust_user	280	320	360	400	490	2100	2300	2400	2400	2400	196	
Aggregated		280	320	360	400	490	2100	2300	2400	2400	2400	196	

SS7

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)
GET	//events?user=locust_user	162	0	3	2000	2000	128.9	2
Aggregated		162	0	3	2000	2000	128.9	2

ABOUT

Traceback (most recent call last):
File "C:\Users\dsrij\cig\sem6\cloudComputing\lab\lab2\PES2UG23CS600\Monolith_CC_Lab-2\CC_Lab-2\venv\lib\site-packages\gevent\ffi\loop.py", line 279, in python_check_callback
def python_check_callback(self, watcher_ptr): # pylint:disable=unused-argument

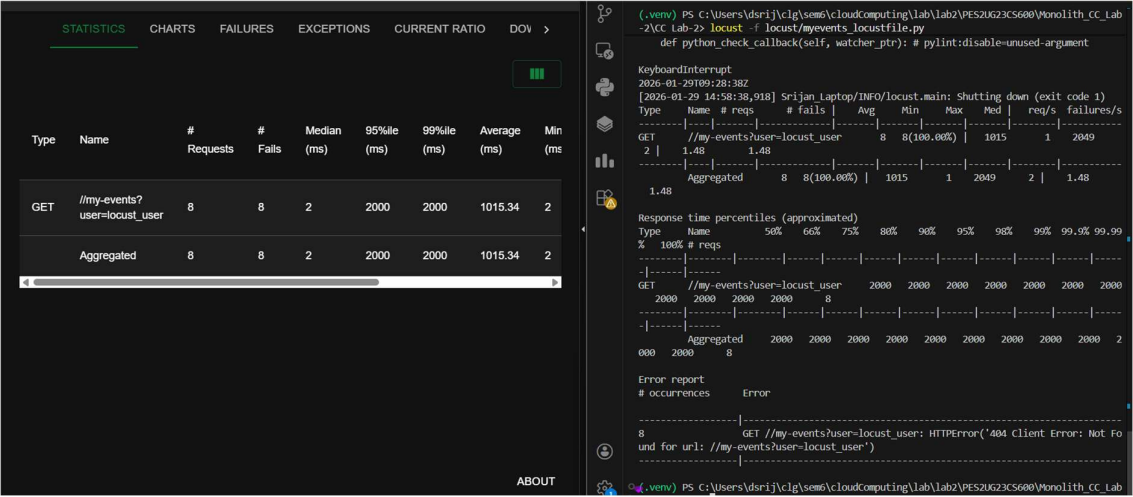
KeyboardInterrupt
2026-01-29T09:32:08Z
[2026-01-29 15:02:08,349] Srijan_Laptop/INFO/locust.main: Shutting down (exit code 0)

Type	Name	# fails	Avg	Min	Max	Med	req/s	failures/s	# reqs
GET	//events?user=locust_user	0(0.00%)	128	1	2051	3	5.42	0.00	162
Aggregated		0(0.00%)	128	1	2051	3	5.42	0.00	162

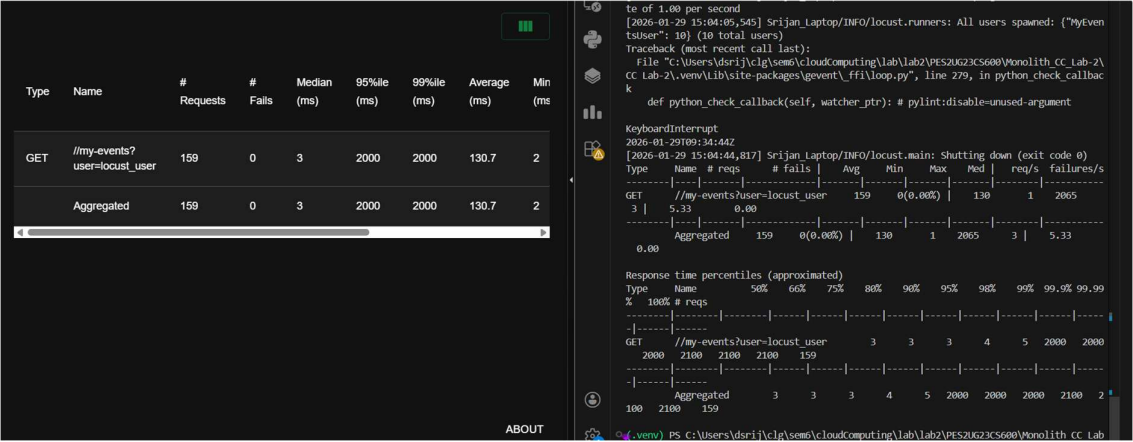
Response time percentiles (approximated)

Type	Name	50%	66%	75%	80%	90%	95%	98%	99%	99.9%	99.99%	100%	# reqs
GET	//events?user=locust_user	3	4	4	4	17	2000	2000	2000	2100	2100	2100	162
Aggregated		3	4	4	4	17	2000	2000	2000	2100	2100	2100	162

SS8



SS9



EXPLANATIONS:

Route: /events

Bottleneck:

The /events route contained an artificial CPU-intensive loop that executed millions of iterations for every request, even though it performed no useful computation.

Change Made:

Removed the unnecessary loop and returned the database query results directly.

Why Performance Improved:

Eliminating wasteful CPU operations reduced request processing time, leading to lower average response time and faster responses under load.

Route: /my-events

Bottleneck:

The /my-events route included an artificial delay loop that performed over a million iterations per request, consuming CPU cycles unnecessarily.

Change Made:

Removed the dummy loop and allowed the route to directly return results from the database query.

Why Performance Improved:

By removing redundant computation, CPU usage per request decreased, which improved response time and overall throughput.

GITHUB: https://github.com/srijan-nodes/cc_lab2_monolith.git